Topic:	*IPC Functions reference summary*
Tools:	*---*
Required skills:	*Basic knowledge of issues related to programming in C language and real-time systems*

Message transfer – MsgSend, MsgReceive, MsgReply

MsgSend

```
int /* status */ MsgSend( int coid,
             const void* smsg,
             int sbytes,
             void* rmsg,
             int rbytes );
```

Arguments are as follows:

- coid - connection id
- smsg - sending data buffer address
- rmsg - received data buffer address
- sbytes - number of bytes to send
- rbytes - max. number of bytes to be received

The function returns:

- value of the **status** argument in MsgReply() - success,
- -1 - error (see `errno` for details)

MsgReceive

```
int /* rcvid */ MsgReceive( int chid,
                void * msg,
                int bytes,
                struct _msg_info * info );
```

Arguments are as follows:

- chid - channel ID; created with ChannelCreate or name_attach(NULL, …)
- msg - received data buffer address
  (big enough to contain at least a pulse – `struct _pulse`)
- bytes - max. number of bytes to be received
- info - additional information about the message

The function returns:

- >0 – rcvid (which encodes the sending thread's ID and a local connection ID)
- 0 – in case of receiving pulse (info isn't updated, msg.type)
- -1 – error (see `errno` for details)

```
struct _msg_info {        /* _msg_info    _server_info */
    uint32_t  nd;         /*  client      server */
    uint32_t  srcnd;      /*  server      n/a */
    pid_t     pid;        /*  client      server */
    int32_t   tid;        /*  thread      n/a */
    int32_t   chid;       /*  server      server */
    int32_t   scoid;      /*  server      server */
    int32_t   coid;       /*  client      client */
    int32_t   msglen;     /*  msg         n/a */
    int32_t   srcmsglen;  /*  thread      n/a */
    int32_t   dstmsglen;  /*  thread      n/a */
    int16_t   priority;   /*  thread      n/a */
    int16_t   flags;      /*  n/a         client */
    uint32_t  reserved;
};
-------------------------------------------------------
struct _pulse {
    uint16_t        type;  // _PULSE_TYPE(0)
    uint16_t        subtype; // _PULSE_SUBTYPE(0)
    int8_t          code;  // _PULSE_CODE_MINAVAIL <= user code <= _PULSE_CODE_MAXAVAIL
    uint8_t         zero[3];
    union sigval    value; // value relevant to the code
    int32_t         scoid; // server connection id
};

union sigval {
    int         sival_int;
    void       *sival_ptr;
};

struct sigevent {
    int                             sigev_notify;
    union {
            int                     __sigev_signo;
            int                     __sigev_coid;
            int                     __sigev_id;
            void                    (*__sigev_notify_function)(union sigval);
            volatile unsigned       *__sigev_addr;
    } __sigev_un1;

    union sigval                    sigev_value;

    union {
            struct {
                    short           __sigev_code;
                    short           __sigev_priority;
            } __st;
            pthread_attr_t          *__sigev_notify_attributes;
            int                     __sigev_memop;
        } __sigev_un2;

};
// don't set sigev_notify directly, instead use:
SIGEV_SET_TYPE(&my_event, SIGEV_PULSE)
if( SIGEV_GET_TYPE(&my_event) == SIGEV_PULSE)
```

MsgReply

```
int MsgReply( int rcvid,
              int status,
              const void* msg,
              int size );
```

Arguments are as follows:

- rcvid - identifier returned by the MsgReceive() function
- status - the value that will be returned by the MsgSend() function
- msg - sending data buffer address
- size - number of bytes to send

The function returns:

- 0 – successful reply delivery
- -1 - error (error code is stored in errno)

Names (named channels)

name_attach

```
name_attach_t * name_attach( dispatch_t * dpp,
                             const char * path,
                             unsigned flags ); // name_attach_t contains chid
```

Arguments are as follows:

- dpp – data structure for the name being created (if NULL, it will be **created**)
- path – the name path to be **registered**
- flags – optional flag NAME_FLAG_ATTACH_GLOBAL for global connections

The function returns:

- pointer to the name_attach_t structure with **channel id (chid)**,
- NULL - error (see *errno* for details)

```
typedef struct _name_attach {
    dispatch_t* dpp;   // dispatch handle used in creation of this connection
    int         chid;  // channel id (MsgReceive)
    int         mntid; // mount id for this name
    int         zero[2];
} name_attach_t;
```

name_open

```
int /* coid */ name_open( const char * name,
                int flags );
```

Arguments are as follows:

- name - the name of the process being located
- flags – optional flag NAME_FLAG_ATTACH_GLOBAL for global connections

The function returns:

- >0 – ID of created connection
- -1 - error

name_detach

```
int name_detach( name_attach_t * attach,
                 unsigned flags );
```

Arguments are as follows:

- attach – structure returned by `name_attach`

The function removes the name from the namespace.

The function returns:

- 0 - when deregistration was successful
- -1 - error

name_close

```
int name_close( int coid );
```

Arguments are as follows:

- coid – connection id returned by `name_open`

The function closes a server connection.

The function returns:

- 0 - when disconnection was successful
- -1 - error

Channels (unnamed)

ChannelCreate

```
int /* chid */ ChannelCreate( unsigned flags );
```

Arguments are as follows:

- chid – channel id of the new channel of calling server process thread

The function creates new channel.

The function returns:

- >0 – (chid) for successful channel creation
- -1 - error

ChannelDestroy

```
int ChannelDestroy( int chid );
```

Arguments are as follows:

- chid – channel id to be destroyed

The function destroys a server channel.

The function returns:

- <>-1 – destroy operation was successful
- -1 - error

ConnectAttach

```
int /* coid */ ConnectAttach( uint32_t nd,
                               pid_t pid,
                               int chid,
                               unsigned index,
                               int flags );
```

Arguments are as follows:

- nd – node descriptor (ND_LOCAL_NODE for local node) on which the channel owner (process) is running
- pid – process ID of the owner of the channel
- chid – channel ID of the channel to be connected

- index - lowest acceptable coid (connection id) ORed with _NTO_SIDE_CHANNEL
- flags - optionally _NTO_COF_CLOEXEC, usually 0

The function connects client to a server channel.

The function returns:
- <>-1 - connection id (coid) when connection was successful
- -1 - error

ConnectDetach

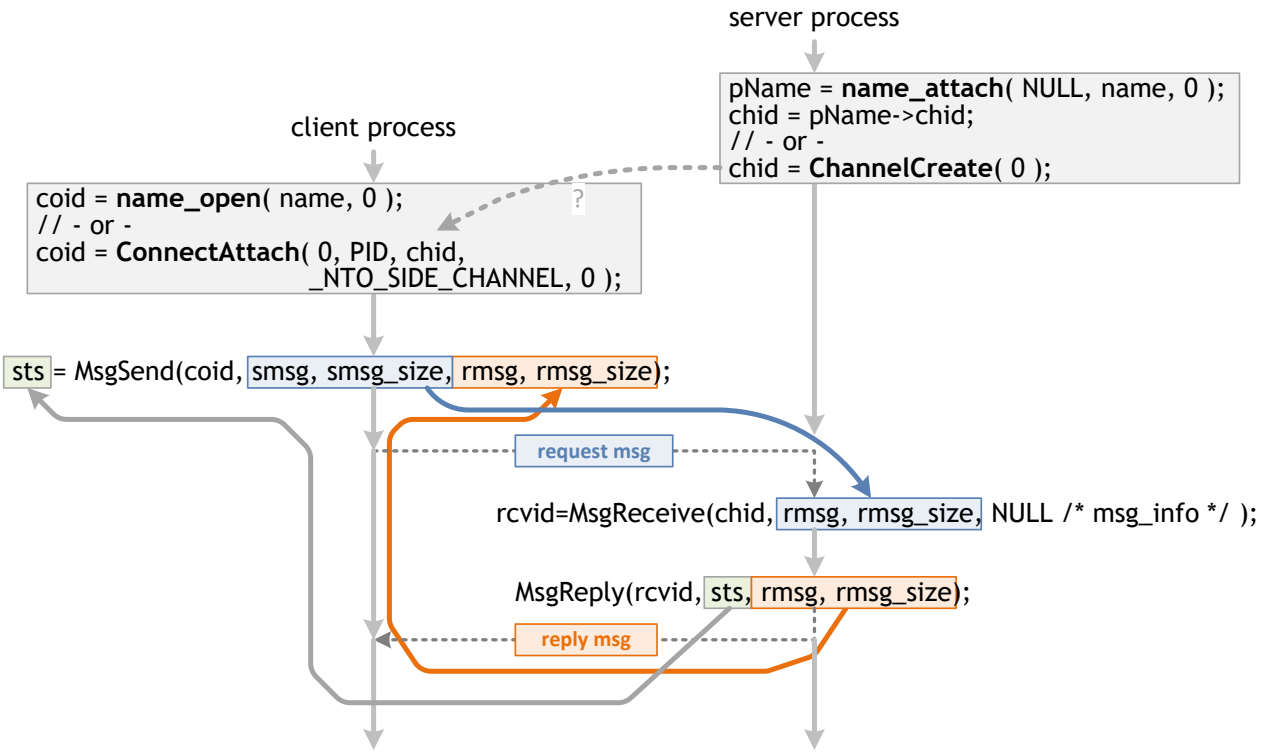
```
int ConnectDetach( int coid );
```

Arguments are as follows:
- coid - connection id returned by `ConnectAttach`

The function closes a server connection.

The function returns:
- 0 - when disconnection was successful
- -1 - error

Architecture of above channel, names and message passing functions



Errors (errno)

This example code allows to use errno code and error message.

```
#include <errno.h>
...
printf("name_attach error! errno = %d (%s)\n", errno, sys_errlist[errno] );
// alternative:
printf("name_attach error! errno = %d (%s)\n", errno, strerror(errno) );
// alternative:
perror("name_attach error!");
```

Includes for above and other useful functions:

```
#include <sys/neutrino.h>      // MsgSend/Receive/Reply
#include <sys/iofunc.h>        // name_attach/detach/open/close
#include <sys/dispatch.h>      // name_attach
#include <sys/netmgr.h>        // ConnectAttach
#incldue <signal.h>            // timer_create
#include <time.h>              // timer_create/delete/settime
```